Complexity

Content

- Concept of complexity
- Theoretical and Practical Complexity
- Complexity: three cases
- Asymptotic notations
- Recurrence relation
- Practical Complexity

Complexity

- Complexity
  - The complexity of an algorithm is **a measure** of the number of **basic operations** the algorithm performs on a **given set of input data**
- Depends on the input data
  - Complexity is a function that depends on **size n** of the input data set
  - Complexity is **significant when n is large**

- Evaluate algorithm complexity to
  - Study of algorithm operation
    - Optimized or not
    - Detect complex parts, slowing down the algorithm
  - Compare different solutions in the same context
    - problem
    - language
    - computer

Time and Memory

- Complexity assessment
  - execution time
  - memory usage
  - ➔ often contradict each other
- We often look for ways to reduce time complexity
  - ignore memory complexity
- Leads to
  - often increases memory complexity
  - increase the "intellectual complexity" of the algorithm
    - Development time
    - Risk of error
    - Maintenance costs

Theoretical and Practical Complexity

- Two methods for assessing time complexity
  - Theoretical method
  - Practical method

- Theoretical method
  - Evaluate the performance of the algorithm when the data size n is very large
  - The performance of the algorithm is evaluated by function of n
  - Without taking into account the details of the algorithm

Theoretical and Practical Complexity

- Practical method
  - Detailed evaluation of the algorithm
  - All statements are considered
  - Evaluate different activities separately
    - addition/subtraction of integers
    - integer multiplication
    - comparison
    - read/write operation
    - memory access
    - …
  - Delivers highly detailed results
    - but often complicated
    - often difficult to compare (due to separate assessments)
  - Detect complex regions
  - Support program optimization

Theoretical and Practical Complexity

- The relationship between theoretical and practical methods
  - Usually only evaluates **set of basic operations**
    - The running time of the algorithm is proportional to the number of these operations
    - For example
      - Number of comparisons and data exchanges for the sorting algorithm
      - Number of additions and multiplications for the matrix multiplication algorithm
      - Number of disk accesses for the database manipulation algorithm
  - Basic operations are assessed separately
  - The choice of the set of basic operations depends
    - data size
    - details to be evaluated
    - analytical ability
    - …

Evaluating complexity

- Based on the structure of the algorithm, we have the following principles for calculating complexity:
  - For basic operations that are **sequential statements**, the number of operations is added together
  - For **conditional statements**, assuming P(X) is the number of basic operations of statement X, then:

    $P(\underline{\text{if}}\ C\ \underline{\text{then}}\ I_1\ \underline{\text{else}}\ I_2) \leq P(C) + \max(P(I_1), P(I_2))$
  - For **loop statements**, the number of basic operations in the loop is $\sum P(i)$, where i is the loop control variable, P(i) is the number of basic operations in the ith iteration

Evaluating complexity

- For function calls
  - If there were no recursive functions, we could always organize them so that each function calls another function whose number of basic operations is already determined
  - If it is **a recursive function**, calculating the number of basic operations usually leads to **a recurrence relation**
    - For example

```
function factorial(n)
begin
 if (n = 0) then
    factorial = 1
 else
    factorial = n * factorial (n-1)
 end if
end
```

Basic operation: multiplication

$C(0) = 0$
$C(n) = C(n-1) + 1$

So: $C(n) = n$

Complexity: three cases

- Complexity depends on the data used by the algorithm

- Best case
  - The data is organized so that the algorithm works most efficiently
  - Minimum value of complexity
- Worst case
  - The data is organized so that the algorithm operates least efficiently
  - Maximum value of complexity
- Average case
  - Data corresponding to the organization is considered average
  - Not simple to determine
  - Usually makes the most sense
  - Not the average of best complexity and worst complexity

Complexity: three cases

- D_n : data set of size n
- $C_A(d)$: complexity of algorithm A for data d of set D_n
- p(d): probability of occurrence of d

- Best case complexity: $Cmin_A(n)$
  - $Cmin_A(n) = \min\{C_A(d), d \in D_n\}$
- Worst case complexity: $Cmax_A(n)$
  - $Cmax_A(n) = \max\{C_A(d), d \in D_n\}$
- Average complexity: $Cavg_A(n)$
  - $Cavg_A(n) = \sum p(d).C_A(d)$

Complexity: three cases

- If the probability of the data occurring is the same, then
  - $card(D_n) = |D_n|$
  - $Cavg_A(n) = (1/|D_n|) \sum C_A(d)$
- Average complexity is generally not possible to be determined precisely
- One thing for sure
  - $Cmin_A(n) \leq Cavg_A(n) \leq Cmax_A(n)$

Complexity: three cases

- Example 1: multiply two matrices

```
matrix_product (A, B, C, n)
begin
 for i from 1 to n
  for j from 1 to n
   C(i,j) = 0
   for k from 1 to n
          C(i,j) = C(i,j) + A(i,k) * B(k,j)
   end for
  end for
 end for
end
```

$Cmin_A(n)$? $Cavg_A(n)$? $Cmax_A(n)$

Complexity: three cases

- Example 2: Searching sequentially in a list

```
function lookup(L, X, n)
begin
 i = 1
 while ((i ≤ n) and (L(i) ≠ X))
          i = i + 1
 end while
 If (i > n) then
          i = 0
 end if
 return i
end
```

Basic operation: comparison

- Best case: $L(1) = X$, $Cmin_A(n) = 1$
- Worst case: X is not in L or $L(n) = X$, $Cmax_A(n) = n$

Complexity: three cases

- Example 2: average complexity
  - q: probability that X is in list L
  - X in L: equal probability of appearing in positions
  - $D_{n,i}$: data set with X at position i, $p(D_{n,i}) = q/n$
  - $D_{n,0}$: data set with X not in L, $p(D_{n,0}) = 1-q$
  - $C_A(D_{n,i}) = i$ and $C_A(D_{n,0}) = n$

$$Cavg_A(n) = \sum_{i=0}^{n} p(D_{n,i}) C_A(D_{n,i}) = (1-q)n + q/n \sum_{i=1}^{n} i$$

$$= (1-q)n + q(n+1)/2$$

Complexity: three cases

- Example 2: average complexity
  - If $q = 1$ (X belongs to L), then $Cavg_A(n) = (n+1)/2$
  - If $q = ½$, then $Cavg_A(n) = (3n+1)/4$

  - Average case
    - X is in the middle of list L, at position n/2 or (n+1)/2
    - Complexity is n/2 or (n+1)/2

Asymptotic notations

- In the general case, we usually do not calculate the exact complexity, but calculate the asymptotic complexity (when n is very large)
- We consider asymptotic notations with respect to n and the change of the function when n is very large

- The asymptotic notations allow
  - evaluating the efficiency of the algorithm
  - comparing the efficiency of algorithms

Asymptotic notations

- Small o
  - f(n) = o(g(n)) when n $\rightarrow \infty$ if:

$$\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < cg(n)$$

  - g(n) is an upper bound of f(n) for **every** constant c
  - f(n) increases slower than g(n):

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

  - For example
    - $n = o(n^2)$
    - $n^2 = o(n^4)$

Asymptotic notations

- Big O
  - f(n) = O(g(n)) when n $\rightarrow \infty$ if:

$$\exists c > 0, \exists \mathrm{n}_0 > 0, 0 \leq f(n) \leq cg(n), \forall n \geq n_0$$

  - g(n) is an upper bound of f(n) for **some** constant c
    - If f = o(g) then f = O(g)
  - f(n) does not increase faster than g(n)
  - For example
    - 2n = O(n^2)
    - $2n^2$ = O(n^2), but $2n^2 \neq$ o(n^2)

Asymptotic notations

- Ω (Omega)
  - f(n) = Ω (g(n)) when n $\rightarrow \infty$ if:

$$\exists c > 0, \exists \mathrm{n}_0 > 0, 0 \leq cg(n) \leq f(n), \forall n \geq n_0$$

  - g(n) is a lower bound of f(n) for **some** constant c
  - f(n) increases at least as fast as g(n)
  - For example
    - $n^2 = \Omega(n)$

Asymptotic notations

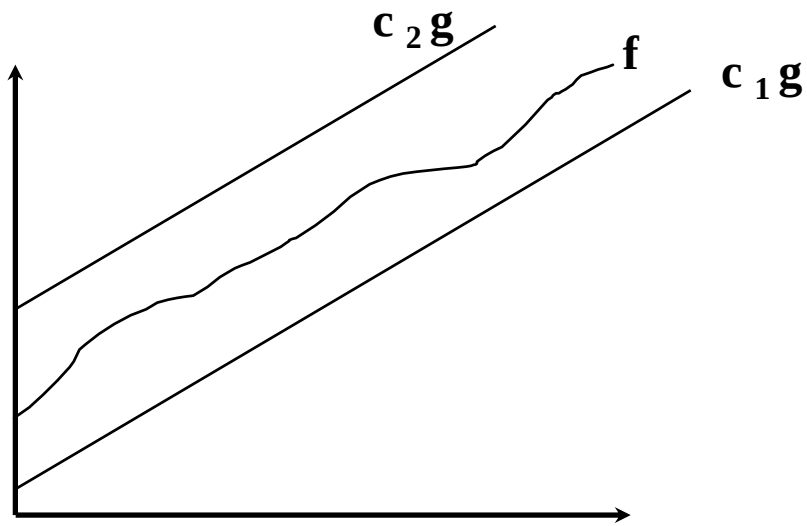
- Θ (Theta)
  - $f(n) = \Theta(g(n))$ when $n \to \infty$ if:

$$\exists c_1 > 0, \exists c_2 > 0, \exists n_0 > 0, 0 \le c_1 g(n) \le f(n) \le c_2 g(n), \forall n \ge n_0$$

  - f(n) and g(n) increase at the same rate (comparable in complexity sense)
  - For example
    - $f(n) = n^2/2 - 3n = n^2(1/2 - 3/n)$, $g(n) = n^2$
    - Determine c_1 and c_2 so that: $c_1 <= 1/2 - 3/n <= c_2$
    - With $c_2 = 1/2$, $n_0 = 12$, $c_1 = 1/2 - 3/12 = 1/4$
    - So $n^2/2 - 3n = \Theta(n^2)$

Asymptotic notations

- Θ (Theta)
  - Definition of upper and lower bound of f(n)



  - If f = Θ(g) then:
    - f = O(g)
    - g = O(f)

Asymptotic notations

- Properties
  - If f = Θ(g) then f = O(g) and f = Ω (g)
    - Deduced from definition
  - If f = Θ(g) then g = Θ(f)
    - Proof

$$\exists(c_1,n_0),\ c_1>0,\ \forall \mathrm{n}>\mathrm{n}_0,\ \ 0\le g(n)\le \frac{1}{c_1}f(n)$$

Asymptotic notations

- Equivalent notation
  - $f \sim g$ when $n \to \infty$ if:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 1$$

  - f(n) and g(n) increase at the same rate (exactly)
  - f(n) and g(n) have the same limit
  - For example
    - $f(n) = n^2 - n$ and $g(n) = n^2$
    - $f(n) \sim g(n)$

Asymptotic notations

- Theoretical complexity when $n \to \infty$

$f = o(g)$	complexity of f < complexity of g
$f = O(g)$	complexity of f $\leq$ complexity of g (in terms of defined constants)
$f = \Theta(g)$	The complexity of f is the same as the complexity of g (in terms of certain constants)
$f = \Omega(g)$	complexity of f $\geq$ complexity of g (in terms of given constants)
$f = \sim(g)$	complexity of f = complexity of g

Asymptotic notations

- Example 1
  - Suppose $C_A(n) = 8n^2 + 2n + 1$ and $C_B(n) = 1000n^2 + 10n + 2$ are functions representing the worst-case running times of two implementations of the same algorithm. Determine the O of the two functions
  - $C_A(n) = 8n^{2+2n} + 1$

    $\leq 8n^2 + 2n^2 + n^2$ with $n \geq 1$

    $= 11n^2$

  choose $c = 11$, $n_0 = 1$, then $C_A(n) = O(n^2)$
  - $C_B(n) = 1000n^2 + 10n + 2$

    $\leq 1000n^2 + 10n^2 + 2n^2$ with $n \geq 1$

    $= 1012n^2$

  choose $c = 1012$, $n_0 = 1$, then $C_B(n) = O(n^2)$
  - Both: $O(n^2)$

Asymptotic notations

- Example 2
  - Prove that $\forall k \in N$: $\sum_{i=1}^{n} i^k = \theta(n^{k+1})$
  - We have: $\sum_{i=1}^{n} i^k \leq \sum_{i=1}^{n} n^k = n.n^k = n^{k+1}$
  - So: with c = 1 $\sum_{i=1}^{n} i^k = O(n^{k+1})$
  - We have: $\sum_{i=1}^{n} i^k \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} i^k \geq \sum_{i=\lceil \frac{n}{2} \rceil}^{n} (\frac{n}{2})^k \geq \frac{n}{2}(\frac{n}{2})^k = \frac{n^{k+1}}{2^{k+1}}$
  - So: with c = $1/2^{k+1}$ $\sum_{i=1}^{n} i^k = \Omega(n^{k+1})$

Recurrence relation

- As shown, calculating the complexity of a recursive function often leads to the recurrence relation
- Common families of recurrence relation
  - First-order recurrence relation
    - linear: $a_n = na_{n-1} + g(n)$
    - non-linear: $a_n = 1/(n + a_{n-1}) + g(n)$

      The first term must have value
  - Quadratic recurrence relation
    - linear: $a_n = ba_{n-1} + ca_{n-2} + g(n)$
    - non-linear: $a_n = 1/(a_{n-1} + a_{n-2}) + g(n)$

      The first two terms must have values
  - k-order recurrence relation
    - linear: $a_n = b_1 a_{n-1} + b_2 a_{n-2} + \ldots + b_k a_{n-k} + g(n)$
    - linear constant coefficient:

      $a_n = b_1 a_{n-1} + b_2 a_{n-2} + \ldots + b_k a_{n-k}$

      The first k terms must have values

Recurrence relation

- Common families of recurrence relations (continued)
  - Full recurrence
    - Linear:
      $a_n = b_1 a_{n-1} + b_2 a_{n-2} + \ldots + b_k a_{n-k} + \ldots + b_{n-1} a_1 + g(n)$
    - non-linear: …

    The first n-1 terms must have values
  - Partition recurrence
    $a_n = ba_{\lfloor n/2 \rfloor} + ca_{\lceil n/2 \rceil} + g(n)$
    or in the form: $a_n = ca_{n/b} + g(n)$, provided n/b is an integer
  - In many cases, it is very complicated to solve the recurrence relation
  - We focus on
    - First-order linear recurrence
    - Second-order linear recurrence with constant coefficients
    - Partition recurrence

Recurrence relation

- First-order linear recurrence (1)
  - Constant coefficient form: $a_n = ba_{n-1} + g_n$, knowing a_0

$a_n = ba_{n-1} + g_n$

$a_{n-1} = ba_{n-2} + g_{n-1}$ * with coefficient b

$a_{n-2} = ba_{n-3} + g_{n-2}$ * with coefficient b^2

…

$a_1 = ba_0 + g_1$ * with coefficient b^{n-1}

$$a_n = b^n a_0 + \sum_{i=1}^{n} g_i \, b^{n-i}$$

Recurrence relation

- First-order linear recurrence (2)
  - Form: $a_n = b_n a_{n-1} + g_n$, knowing a_0

Let: $a_n = b_1 b_2 \ldots b_n y_n$, $a_0 = y_0$

So: $b_1 b_2 \ldots b_n y_n = b_1 b_2 \ldots b_n y_{n-1} + g_n$

Or: $y_n = y_{n-1} + g_n /(b_1 b_2 \ldots b_n)$

Leading to: $$y_n = y_0 + \sum_{i=1}^{n} \frac{g_i}{b_1 b_2 \ldots b_i}$$

Finally: $$a_n = \prod_{i=1}^{n} b_i \left(a_0 + \sum_{i=1}^{n} \frac{g_i}{b_1 b_2 \ldots b_i}\right)$$

Recurrence relation

- Second-order linear recurrence with constant coefficients
  - Form: $a_n = ba_{n-1} + ca_{n-2}$ (1)
  - The solution of the above formula has the form $a_n = r^n$, r is a constant
  - That is: $r^n = br^{n-1} + cr^{n-2}$
  - Divide both sides of the formula by r^{n-2}, we have:

    $r^2 = br + c$
  - Then $a_n = r^n$ is the solution of the recurrence relation when r is the solution of the equation

    $$r^2 - br - c = 0 \quad (2)$$

  (*called the characteristic equation of the recurrence relation*)
  - In the general case, we obtain two distinct solutions r_1 and r_2
  - Then, $a_n = \alpha r_1^n + \beta r_2^n$ is also a solution of (1), with α and β being constants
    - Prove…

Recurrence relation

- Second-order linear recurrence with constant coefficients
  - $a_n = \alpha\, r_1^n + \beta\, r_2^n$
  - The coefficients α and β are determined by the initial conditions

    $\alpha + \beta = a_0$

    $\alpha r_1 + \beta r_2 = a_1$

  - For example
    - Find the solution of the recurrence relation $a_n = a_{n-1} + 2a_{n-2}$ with initial conditions $a_0 = 2$ and $a_1 = 7$
    - Characteristic equation of the recurrence relation: $r^2 - r - 2 = 0$
    - Solutions of the characteristic equation: $r_1 = 2$ and $r_2 = -1$
    - So $a_n = \alpha\,(2)^n + \beta\,(-1)^n$
    - Substituting the initial conditions, we have

      $a_0 = \alpha + \beta = 2$

      $a_1 = 2\alpha - \beta = 7$
    - So: $\alpha = 3$, $\beta = -1$
    - Solution of the recurrence relation: $a_n = 3.2^n - (-1)^n$

Recurrence relation

- Second-order linear recurrence with constant coefficients
  - If the characteristic equation has a double solution, then the solution of (1) is:

    $a_n = (\alpha\, n + \beta)r^n$
  - For example
    - Find the solution of the recurrence relation $a_n = 4a_{n-1} - 4a_{n-2}$ with initial conditions $a_0 = 3$ and $a_1 = 8$
    - Characteristic equation of the recurrence relation: $r^2 - 4r + 4 = 0$
    - The characteristic equation has a double root: $r = 2$
    - So $a_n = (\alpha n + \beta)(2)^n$
    - Substituting the initial conditions, we have

      $a_0 = 3 = \beta$

      $a_1 = (\alpha + \beta)2 = 8$
    - So: $\alpha = 1$, $\beta = 3$
    - Solution of the recurrence relation: $a_n = n2^n + 3.2^n$

Recurrence relation

- Partition recurrence (1)
  - Recurrence of the form: $a_n = ba_{\lfloor n/2 \rfloor} + ca_{\lceil n/2 \rceil} + g(n)$
  - Analyze the problem into two sub-problems
    - size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$
    - So, usually $b = c$ (= 1)
    - $g(n)$: cost of analyzing and combining sub-solutions
  - When n is large, it can be seen that
    $\lfloor n/2 \rfloor = \lceil n/2 \rceil = n/2$
  - So, the recurrence becomes
    $a_n = 2\, a_{n/2} + g(n)$

Recurrence relation

- Partition recurrence (2)
  - Usually considered in the form:
    $a_n = ca_{n/b} + g(n)$, with $n=b^k$
  - Or: $a_{b^k} = c\ a_{b^{k-1}} + g(b^k)$
  - Put $t_k = a_{b^k}$
  - So we have: $t_k = ct_{k-1} + g(b^k)$
    $t_0 = 1$ (= $a_1 = 1$)

    $t_k = ct_{k-1} + g(b^k)$
    $t_{k-1} = ct_{k-2} + g(b^{k-1})$
    …
    $t_1 = ct_0 + g(b^1)$

Recurrence relation

- Partition recurrence (3)

$t_k = ct_{k-1} + g(b^k)$

$t_{k-1} = ct_{k-2} + g(b^{k-1})$ $\quad *c$

$t_{k-2} = ct_{k-3} + g(b^{k-2})$ $\quad *c^2$

…

$t_1 = ct_0 + g(b^1)$ $\quad *c^{k-1}$ $\quad t_0 = 1$

$$t_k = c^k + \sum_{i=0}^{k-1} c^i g(b^{k-i})$$

with $n = b^k \Leftrightarrow k = \log_b n$

$b^{k-i} = n/b^i$

$$c^{\log_b n} = n^{\log_b c}$$

$$\boxed{a_n = n^{\log_b c} + \sum_{i=0}^{k-1} c^i g(\frac{n}{b^i})}$$

Recurrence relation

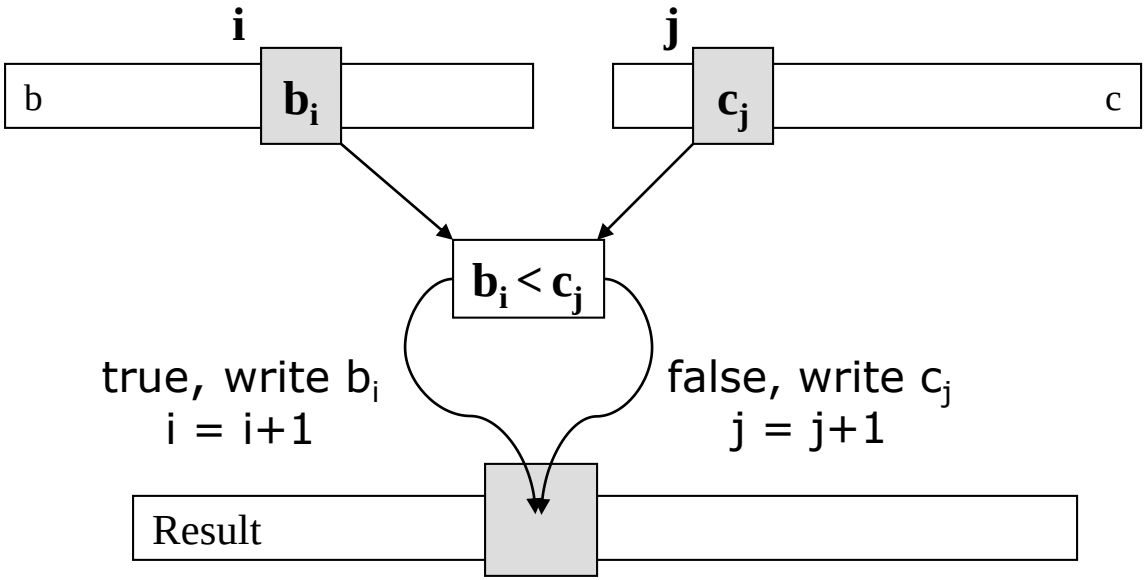
- Merge sort (1)
  - Calculate the complexity of merge sort algorithm

```
merge-sort (A, p, q)
Begin
 If (p < q) then
      r = (p+q)/2
      merge-sort (A, p, r)
      merge-sort (A,r + 1, q)
      merge (A, p, r, q)
 EndIf
End
```

  - Divide into two subproblems of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$

Recurrence relation

- Merge sort (2)
  - Merge two sorted lists b and c of size n/2: merge()



  - Use at most n-1 comparisons

Recurrence relation

- Merge sort (3)
  - So we have the recurrence relation ($a_n = C(n)$)

    $a_n = 2\, a_{n/2} + n - 1$

$$a_n = n^{\log_b c} + \sum_{i=0}^{k-1} c^i g\left(\frac{n}{b^i}\right) \Rightarrow a_n = n^{\log_2 2} + \sum_{i=0}^{k-1} 2^i \left(\frac{n}{2^i} - 1\right)$$

use formula: $\sum_{i=0}^{n} q^i = \frac{1-q^{n+1}}{1-q} \quad (q \neq 1)$

$$a_n = n + k\,n - \sum_{i=0}^{k-1} 2^i = n + k\,n - (2^k - 1)$$ with $k = \log_2 n$

$$a_n = n + n\log_2 n - (2^{\log_2 n} - 1)$$ use $c^{\log_b^n} = n^{\log_b^c}$

$$\boxed{a_n = n\log_2 n + 1}$$

So: $C(n) = \Theta(n\log(n))$

Practical complexity

- Detailed evaluation of the algorithm
- All statements are considered
- Evaluate different types of operations separately
  - addition/subtraction of integers
  - integer multiplication
  - comparison
  - read/write operation
  - memory access
  - …
- Delivers highly detailed results
  - but often complicated
  - difficult to compare (due to separate assessments)

Practical complexity

- Insertion sort (1)
  - Evaluate the complexity of insertion sort algorithm
  - Each statement is assigned a cost in terms of running time and number of times it is executed (times). For each j = 2,3,…n, t_j is the number of times the While statement is executed.

insertion-sort(A)	cost	times
Begin		
For j from 2 to n	c_1	n
key = A[j]	c_2	n-1
i = j – 1	c_3	n-1
While (i > 0 and A[i] > key) do	c_4	$\sum_{j=2}^{n} t_j$
A[i+1] = A[i]	c_5	$\sum_{j=2}^{n} (t_j - 1)$
i = i – 1	c_6	$\sum_{j=2}^{n} (t_j - 1)$
EndWhile		
A[i+1] = key	c_7	n-1
EndFor		
End		

Practical complexity

- Insertion sort (2)
  - Total running time is

$$C(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} t_j + c_5 \sum_{j=2}^{n} (t_j - 1) + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7(n-1)$$

  - Best case complexity: the list is sorted, then $t_j = 1$ for all j. So:

$$C(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 (n-1) + c_7 (n-1)$$
$$= n(c_1 + c_2 + c_3 + c_4 + c_7) - (c_2 + c_3 + c_4 + c_7)$$

We can write $C(n) = an + b$, where a and b are constants

The best case complexity is a linear function of n

Practical complexity

- Insertion sort (2)
  - Worst case complexity: the list is sorted in reverse order, then $t_j = j$ for all j.
  - We have:
  - So: $\sum_{j=1}^{n} j = \frac{n(n+1)}{2}, \sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1, \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$

$$C(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n(n+1)}{2} - 1\right) + c_5\left(\frac{n(n-1)}{2}\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7(n-1)$$

$$= \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right)n^2 + \left(c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7\right)n - \left(c_2 + c_3 + c_4 + c_7\right)$$

We can write $C(n) = an^2 + bn + c$, where a, b and c are constants

Worst case complexity is function of n^2

Practical complexity

- Insertion sort (3)
  - Average case complexity
    - Suppose we apply the algorithm to a list of n randomly selected elements
    - What is the value of t_j? Where is the element A[j] inserted in the list A[1..j-1]?
    - In the average case, half of the elements of A[1..j-1] are greater than A[j] and half of the elements of A[1..j-1] are less than A[j]
    - So $t_j = j/2$
  - Similarly in the worst case, C(n) will be of the form $C(n) = an^2 + bn + c$, where a, b and c are constants
  - The average case complexity is a function of n^2

Practical complexity

- Insertion sort (4)
  - However, usually what we are really interested in are the values of the asymptotic notations (usually O and Θ)
  - Only the significant term of the complexity expression is significant when n is very large, ignoring the remaining terms
  - When n is large, we always ignore the constant of the important term
  - So the complexity of insertion sort algorithm in
    - best case: $\Theta(n)$
    - worst case: $\Theta(n^2)$
    - average case: $\Theta(n^2)$

Algorithm classes by complexity

- **Algorithms are often compared by their worst-case complexity**, i.e. the value of O(g(n))
- Algorithm classes by complexity
  - Constant: O(1), all instructions execute exactly once regardless of data size
  - Under linearity: O(log(n)), the size of the problem is subdivided by a constant at each iteration step
  - Linear:
    - O(n), loop n times, loop body performs work independently of n
    - O(nlog(n)), the size of the problem is subdivided by a constant at each iteration, and at each subdivision a linear traversal of the data is performed
  - Polynomial: $O(n^k)$, algorithms are considered slow when $k>3$
  - Exponential function: $O(k^n)$, $k\geq 2$, algorithms are not applicable
  - **Usable algorithms** usually have complexity $\leq O(n^3)$

Algorithm classes by complexity

- Illustration of complexity of algorithm classes

n	$\log_2(n)$	$n\log_2(n)$	n^2	n^3	2^n
1	0	0	1	1	2
10	3.32	33.2	100	10^3	1024
100	6.64	664	10^4	10^6	1.26^{30}
1000	9,965	9 965	10^6	10^9	∞
10^6	19,931	19 931 568	10^{12}	10^{18}	∞

Exercises

- Problem 1
  a. Show that $f(n) = n^3 - n^2\log(n) + 100 = \Theta(n^3)$
  b. Show that $n^k a_k + n^{k-1}a_{k-1} + \ldots + na_1 + a_0 = O(n^k)$
  c. $f(n) = \log(2n + a)$, prove that $f(n) = \Theta(\log(n))$

Exercises

- Problem 2
  - Given the following algorithms with their running times:

    O(nlogn)

    $O(n^2)$

    $O(2^n)$

    $O(\sqrt{n})$
  - Rank them based on their efficiency as n grows

Exercises

- Problem 3
  - Evaluate the running time of Bubble sort algorithm in three cases

```
BUBBLESORT(A)
for i = 1 to A.length − 1
    for j = A.length downto i + 1
        if A[j] < A[j − 1]
            exchange A[j] with A[j − 1]
```

Exercises

- Problem 4

The following code fragment implements Horner's rule for evaluating a polynomial

$$\begin{aligned} P(x) &= \sum_{k=0}^{n} a_k x^k \\ &= a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x a_n) \cdots)) \ , \end{aligned}$$

given the coefficients $a_0, a_1, \ldots, a_n$ and a value for x:

```
y = 0
for i = n downto 0
    y = a_i + x · y
```

a. In terms of Θ-notation, what is the running time of this code fragment for Horner's rule?

b. Write pseudocode to implement the naive polynomial-evaluation algorithm that computes each term of the polynomial from scratch. What is the running time of this algorithm? How does it compare to Horner's rule?

Exercises

- Problem 5
  - Evaluate the complexity of the following algorithm

```
Binary-search (A, x, l, r)
// find x in the sorted list A[l,r]
begin
    if (l = r) then return (l)
    else
        m = (l + r)/2
        if (x ≤ A[m]) then return (Binary-search(A, x, l, m))
        else return (Binary-search(A, x, m+1, r))
        endif
    endif
end
```